



FPGA Implementation of a Parameterized FIFO-Based Communication Buffer with Fault Injection Analysis

A Technical Report

Submitted by

Avinash Tanti (252CN034)

In partial fulfillment of the requirements for the degree of

MASTER OF TECHNOLOGY

in

Electronics and Communication Engineering

Under the guidance of

Prof. Prashantha Kumar

Department of Electronics and Communication Engineering
National Institute of Technology Karnataka

April 2026

Abstract

In modern digital systems, reliable data transfer between independently operating modules is a critical design requirement, particularly in the presence of rate mismatch between data producers and consumers. This work presents the design, simulation, and FPGA implementation of a parameterized synchronous FIFO-based communication buffer using Verilog HDL, with emphasis on system-level behavior, flow control, and communication reliability.

The proposed architecture extends a conventional FIFO design by incorporating a valid-ready handshake protocol, enabling controlled data transfer and effective backpressure management. A modular design approach is adopted, separating the FIFO core from the communication control logic, thereby improving scalability, reusability, and clarity of design.

A key contribution of this work is the introduction of a controlled fault injection mechanism, which allows intentional violation of handshake conditions to analyze system behavior under improper operation. This enables a direct comparison between incorrect and correct communication scenarios within a unified framework. Simulation results demonstrate that violation of handshake protocols leads to data corruption, invalid read/write operations, and loss of ordering, whereas proper protocol enforcement ensures stable and reliable data transfer.

The design is synthesized and implemented on a Cyclone V FPGA using the Quartus Prime toolchain, and validated on a DE1-SoC development board. Hardware results are consistent with simulation observations, confirming correct FIFO operation in normal mode and observable data inconsistencies under fault conditions. Performance analysis indicates a latency of approximately one to two clock cycles and a peak throughput of one data word per clock cycle under continuous valid-ready conditions.

The proposed framework provides both functional verification and practical insight into the importance of protocol-driven communication, making it suitable for FPGA-based system design and reliability analysis.

CHAPTER 1: Introduction and Objectives

Introduction

In modern digital systems, data transfer between independently operating modules often occurs under rate mismatch conditions, leading to issues such as data loss, overflow, and violation of ordering. FIFO buffers are commonly used to address this by temporarily storing data and preserving sequence.

However, a conventional FIFO primarily provides storage and does not ensure reliable communication between modules. Practical systems require coordination between sender and receiver using handshake-based protocols such as the valid-ready mechanism, enabling controlled data transfer and backpressure handling.

Most FIFO systems are evaluated under ideal conditions without analyzing behavior under protocol violations. To address this, this work presents a FIFO-based communication buffer integrating handshake-driven flow control and a controlled fault injection mechanism, verified through simulation and FPGA-based hardware validation.

Objectives

The main objective of this work is to design and validate a FIFO-based communication system with improved reliability and system-level behavior analysis. The specific objectives are:

- To design a parameterized synchronous FIFO using pointer-based circular buffer architecture.
- To integrate a valid-ready handshake protocol for controlled data transfer.
- To develop a modular architecture separating FIFO core and communication logic.
- To implement a fault injection mechanism to analyze protocol violations.
- To verify functionality through simulation and waveform analysis.

CHAPTER 2: System Architecture

System Overview

The proposed system is designed as a communication framework for reliable data transfer between modules operating at different rates. It follows a producer–buffer–consumer model, where a FIFO-based communication buffer regulates data flow and maintains ordering.

The system consists of three main components:

- Traffic Generator (Producer)
- FIFO Communication Buffer
- Traffic Sink (Consumer)

The generator produces data continuously, while the sink consumes data at varying rates, creating realistic rate mismatch conditions. The FIFO buffer temporarily stores incoming data and releases it when the consumer is ready, ensuring correct sequencing and controlled transfer.

Modular Architecture

The system is implemented using a modular RTL design approach, dividing the communication buffer into two layers:

- **FIFO Core:** Implements memory storage, pointer logic, and full/empty detection.
- **Communication Wrapper:** Implements valid–ready handshake and controls read/write operations.

This separation improves design clarity, reusability, and simplifies verification.

RTL Implementation Overview

The complete system is implemented using a hierarchical structure. The top-level module, `fifo_system_top`, integrates the communication buffer with input and output interfaces.

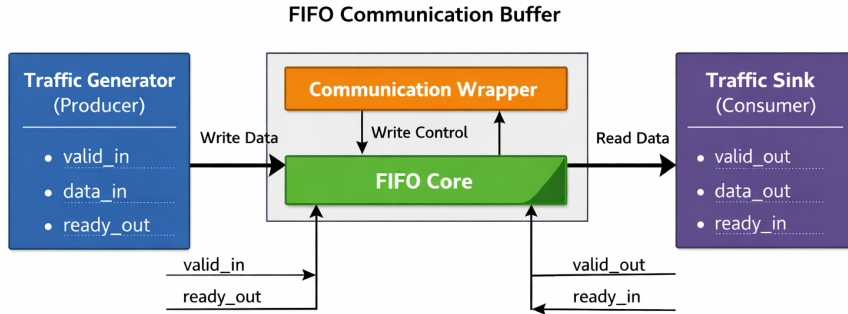


Figure 1: System Architecture

Internally, the communication buffer module (`fifo_comm_buffer`) connects the FIFO core (`fifo_core`) with handshake control logic. The wrapper translates handshake signals into safe read and write enable signals based on FIFO status conditions.

Data is written into the FIFO when both `valid_in` and `ready_out` are asserted, and read when both `valid_out` and `ready_in` are asserted. This ensures synchronized data transfer and prevents invalid operations.

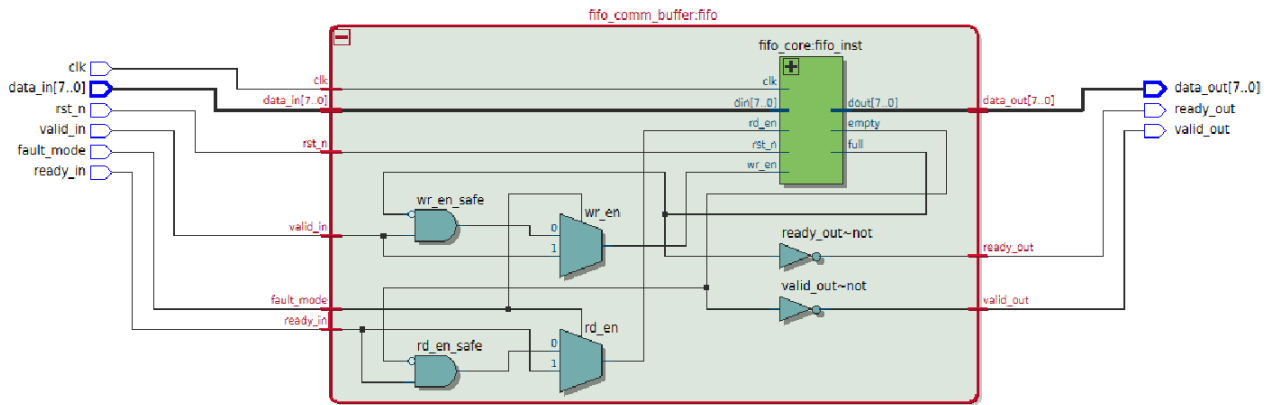


Figure 2: RTL Block Diagram of FIFO Communication System

CHAPTER 3: FIFO and Communication Design

FIFO Core Design

The FIFO core is implemented as a parameterized synchronous design using a circular buffer structure. Data is stored in a memory array of depth N and width W , allowing flexible configuration based on system requirements. The circular buffer enables efficient memory reuse without requiring data shifting.

Data flow within the FIFO is controlled using two pointers:

- **Write Pointer** (*wr_ptr*): Indicates the next location for data insertion.
- **Read Pointer** (*rd_ptr*): Indicates the next location for data retrieval.

The pointers increment on valid operations and wrap around using modulo arithmetic. FIFO status is determined using pointer comparison, where equality indicates an empty condition and a one-step offset indicates a full condition. These conditions prevent invalid read and write operations.

Handshake-Based Communication

To enable reliable data transfer, the FIFO is integrated with a valid-ready handshake protocol. On the input side, **valid_in** indicates the availability of data, while **ready_out** indicates the FIFO's ability to accept new data. A write operation occurs only when both signals are asserted.

Similarly, on the output side, **valid_out** indicates the availability of data, while **ready_in** indicates that the consumer is ready. A read operation occurs only when both signals are asserted.

This handshake mechanism provides flow control and ensures that data transfer occurs only under valid conditions, preventing overflow and underflow.

RTL Implementation of Control Logic

At the RTL level, the handshake signals are translated into safe enable signals for the FIFO core. The write and read enable signals are generated as:

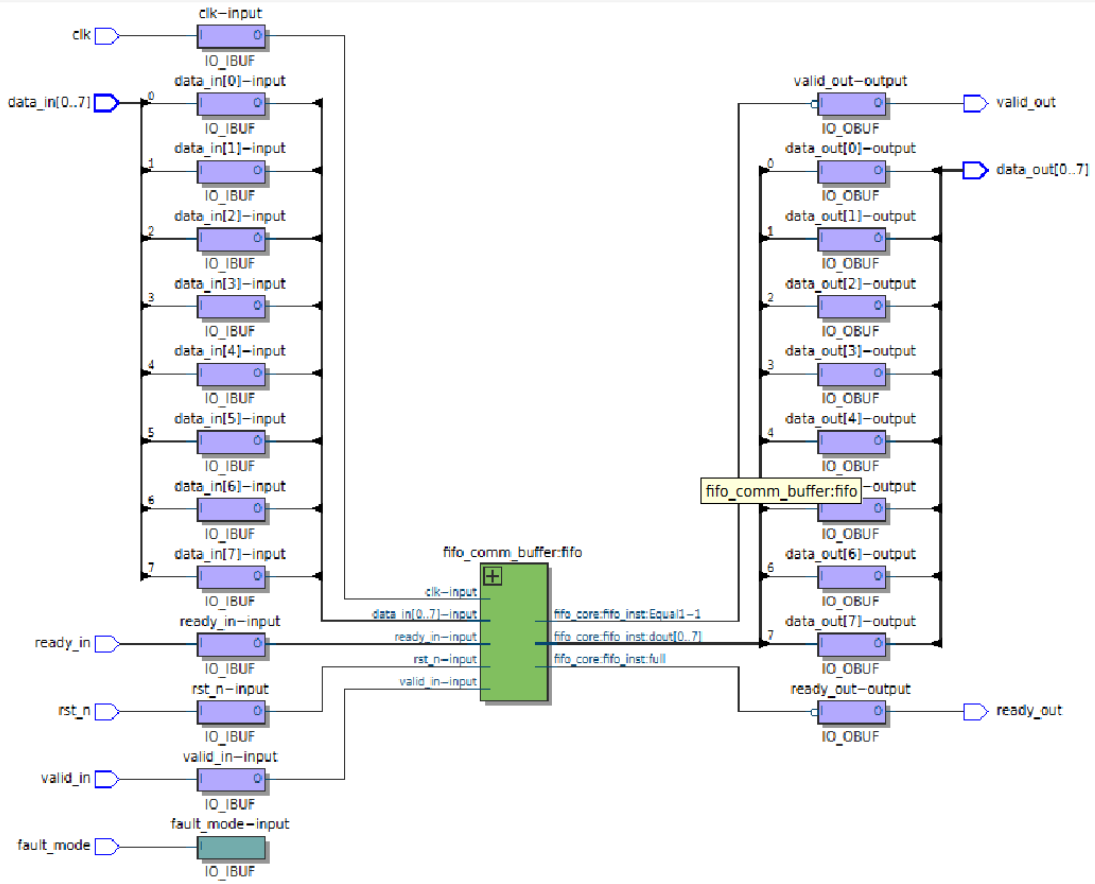


Figure 3: RTL Implementation of Communication Wrapper and Control Logic

$$wr_en_safe = valid_in \wedge \neg full$$

$$rd_en_safe = ready_in \wedge \neg empty$$

These signals ensure that write operations are blocked when the FIFO is full and read operations are blocked when it is empty. This protects the system from invalid data transactions and maintains correct data ordering.

CHAPTER 4: Fault Injection and Simulation Analysis

Fault Injection Methodology

To evaluate system robustness, a controlled fault injection mechanism is introduced using the signal `fault_mode`, which switches between normal and faulty operation.

In normal mode, data transfer follows the valid-ready protocol:

$$wr_en = valid_in \wedge \neg full, \quad rd_en = ready_in \wedge \neg empty$$

In fault mode, FIFO protection is bypassed:

$$wr_en = valid_in, \quad rd_en = ready_in$$

This permits writes when full and reads when empty, intentionally violating correct operation. The switching is implemented using multiplexers to generate deterministic faulty scenarios.

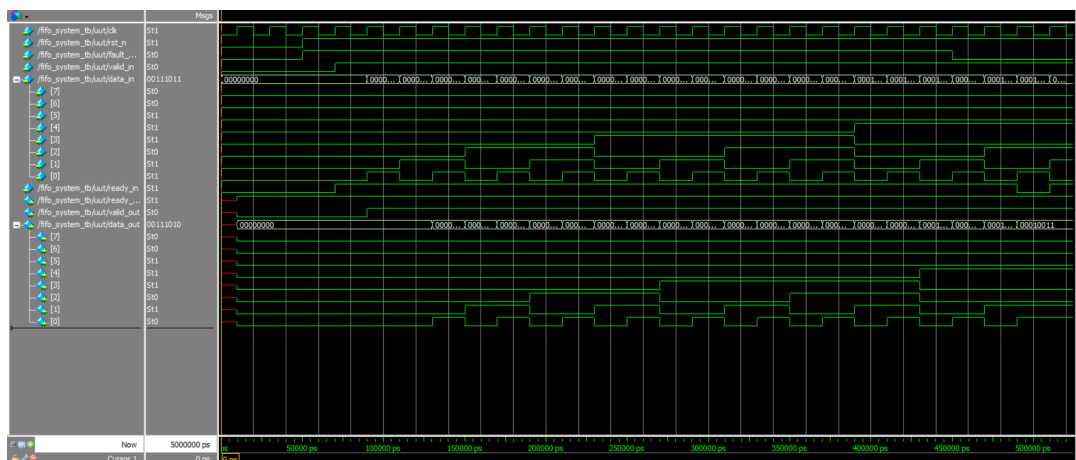
Simulation Setup

The design is verified in ModelSim using a testbench that drives input data and handshake signals. The simulation includes both fault and normal modes within a single run, enabling direct comparison of system behavior.

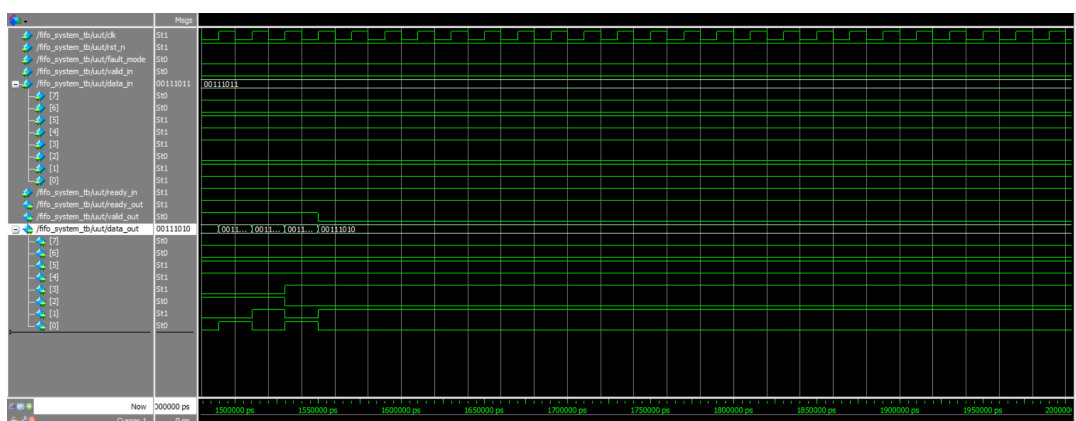
In fault mode, uncontrolled operations lead to data corruption, invalid reads, and loss of ordering. The output does not match the input sequence due to violation of control conditions.

In normal mode, handshake enforcement ensures correct sequencing and stable data transfer, with output matching input after a fixed delay.

Simulation Results



(a) Fault Mode Behavior



(b) Normal Mode Behavior

Figure 4: Simulation waveform showing (a) Fault Mode and (b) Normal Mode behavior

Performance Analysis

Latency, defined as the delay between input and corresponding output, is approximately 1–2 clock cycles from the waveform.

Throughput, defined as the rate of valid data transfer, reaches one data word per clock cycle under continuous valid–ready conditions, while becoming irregular in fault mode.

These results confirm that system correctness and performance depend on proper handshake-based flow control rather than FIFO storage alone.

CHAPTER 5: FPGA Implementation and Hardware Validation

FPGA Platform

The proposed FIFO-based communication system is implemented on the DE1-SoC development board, which features a Cyclone V FPGA. The platform provides configurable logic resources, on-board switches for input control, and LEDs for output observation, enabling real-time validation of the design.

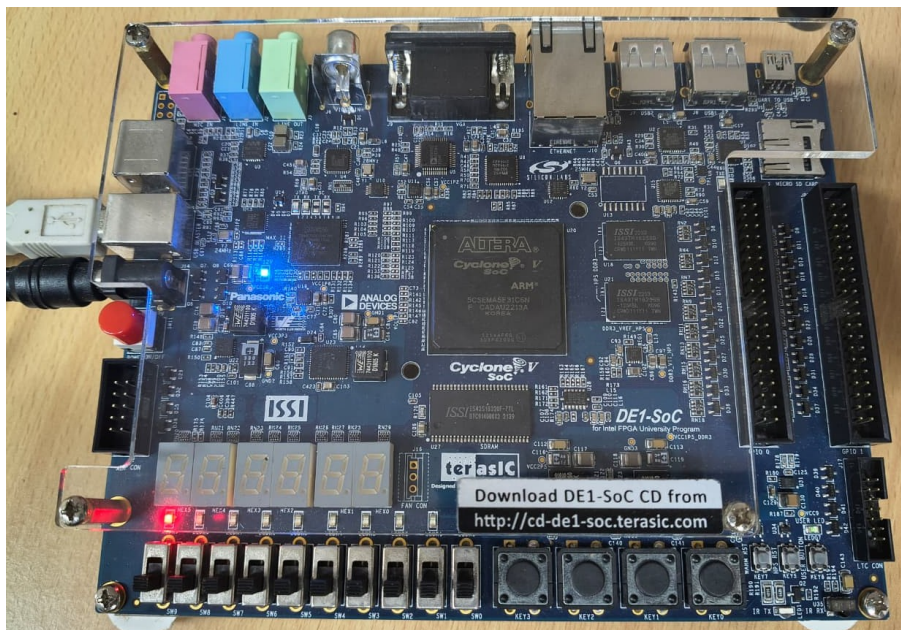


Figure 5: DE1-SoC FPGA Development Board

Design Flow

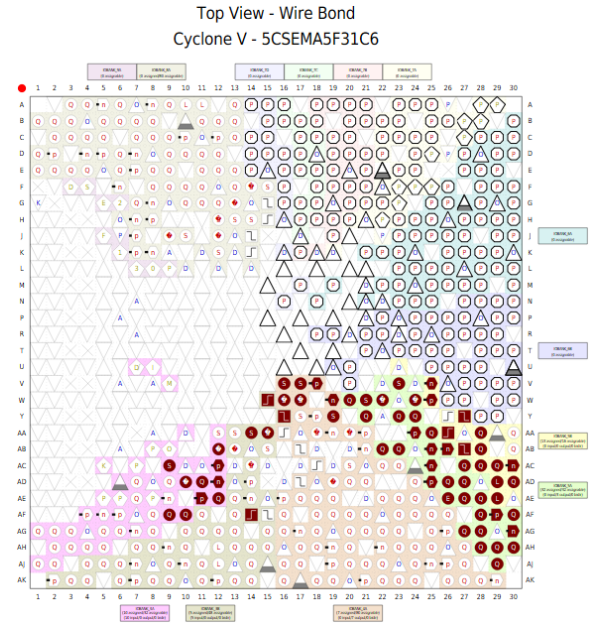
The complete design flow includes RTL design using Verilog HDL, functional simulation using ModelSim, synthesis and implementation using Quartus Prime, and hardware programming via JTAG interface.

Resource Utilization

The synthesized design utilizes minimal FPGA resources, indicating an efficient implementation. The design primarily uses logic elements and

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sun Mar 29 20:13:38 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	fifo_system_top
Top-level Entity Name	fifo_system_top
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	21 / 32,070 (< 1 %)
Total registers	35
Total pins	23 / 457 (5 %)
Total virtual pins	0
Total block memory bits	128 / 4,065,280 (< 1 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

(a) Quartus Compilation Summary



(b) FPGA Pin Mapping Configuration

Figure 6: FPGA Design Implementation Results

registers, with no requirement for DSP blocks or specialized hardware resources. This demonstrates that the system is lightweight and suitable for integration into larger digital systems.

Pin Configuration

The input and output signals are mapped to the FPGA board using the Pin Planner tool. Switches are used to provide input data and control signals such as `valid_in` and `fault_mode`, while LEDs are used to display output data and status signals such as `valid_out` and `ready_out`.

FPGA Programming

The design is programmed onto the FPGA using the Quartus Programmer via JTAG interface. The compiled configuration file (.sof) is successfully loaded onto the target device, confirming correct device selection and programming setup.

Hardware Results

The system behavior is validated on hardware by applying different input conditions using switches and observing the output through LEDs.

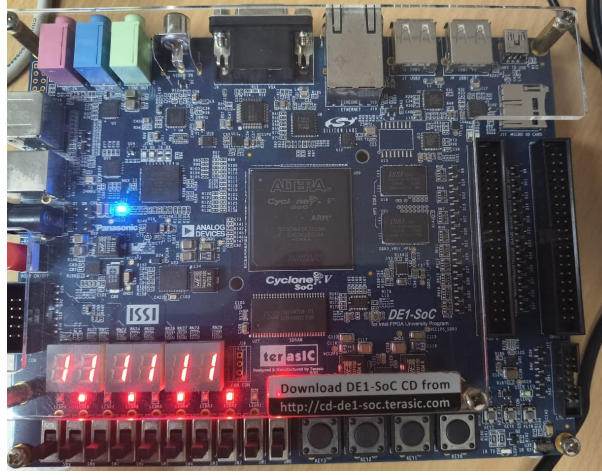
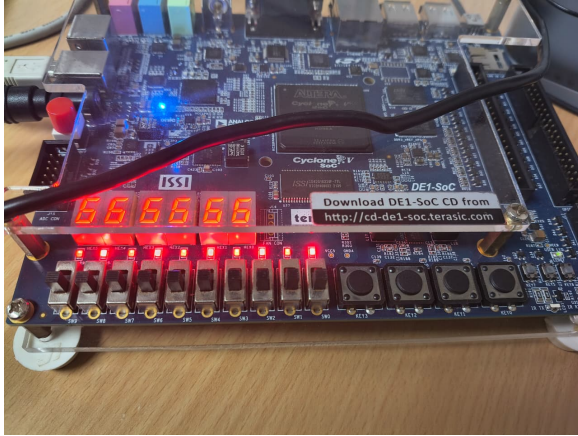
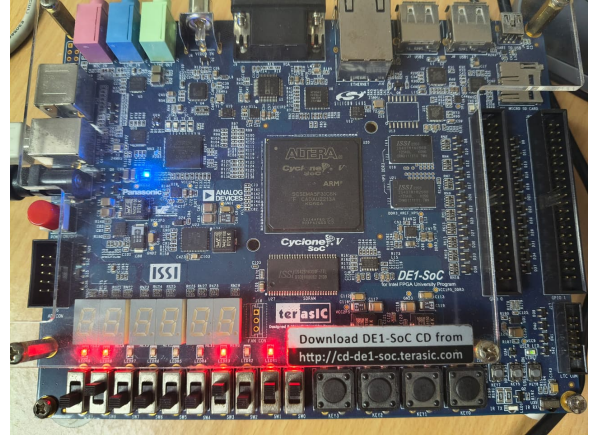


Figure 7: FPGA Programming via JTAG Interface



(a) Initial Output State



(b) Output During Operation

Figure 8: FPGA Hardware Output Observation

In normal mode, the FIFO operates correctly, maintaining proper data ordering and stable output. The LED outputs reflect the expected behavior based on the applied inputs.

In fault mode, the output exhibits irregular behavior due to violation of handshake conditions, demonstrating data inconsistency and incorrect transfer. This behavior matches the simulation results and confirms the effect of protocol violations in real hardware.

Validation Summary

The hardware implementation results closely match the simulation observations, validating the correctness of the RTL design. The system successfully demonstrates both correct operation under normal conditions and faulty behavior under protocol violations.

References

1. D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed., Morgan Kaufmann, 2012.
2. M. M. Mano and M. D. Ciletti, *Digital Design*, 5th ed., Pearson, 2013.
3. S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed., Prentice Hall, 2003.
4. IEEE Standards Association, *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005.
5. Intel Corporation, *Quartus Prime Standard Edition User Guide: Design Compilation*, 2023.
6. Intel Corporation, *Cyclone V Device Handbook*, 2022.
7. Terasic Technologies, *DE1-SoC User Manual*, 2021.
8. W. J. Dally and J. W. Poulton, *Digital Systems Engineering*, Cambridge University Press, 1998.
9. J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2017.
10. S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed., McGraw-Hill, 2013.
11. Intel Corporation, *Introduction to FPGA Design using Quartus Prime*, Intel FPGA Training Material, 2022.
12. ARM Ltd., *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022E, 2013.
13. P. Alfke, “Efficient Shift Registers, LFSR Counters, and Long Pseudo-Random Sequence Generators,” *Xilinx Application Note*, 2007.